





## PRIORYTET OPERATORÓW

BINARNA  
PROMIENNA  
EL. TAB.

$t[i]^{\wedge} = t[j]; t[j]^{\wedge} = t[i]; t[i]^{\wedge} = t[j];$

|                      |                                                 |
|----------------------|-------------------------------------------------|
| WYRAŻENIA            | arr, a--, [], ., →                              |
| UNARNE (tylko jedno) | *, &, +, -, !, ~, ++a, --a, sizeof, (notowanie) |
| BINARNE              | * / % [arytmetyczne]                            |
|                      | + -                                             |
|                      | <<, >>                                          |
|                      | >, <, <=, >= [porównawcze]                      |
|                      | ==, !=                                          |
|                      | &                                               |
|                      | ^ [bitowe]                                      |
|                      |                                                 |
|                      | &&                                              |
|                      |                                                 |
| WARUNKOWY            | ?: [skrótowy IF]                                |
| PRZYPISANIA          | =, *=, &= ... etc.                              |

← np. &a - adres a itd.  
(wymagają tylko jednego składnika)

## TABLICE I INNE TAKIE

! C nie, odwołanie się do el. poza tab nie będzie ważne za bTgd

! int tab[4] = {1, 2, 3}; - skutkuje wypełnieniem reszty 0  
HACK 1: sizeof tab / sizeof tab[0] = l. elementów tab

[ - char tab - po ostatnim el. zawsze '\0'

Wielowymiarowe: char t[4][3];  
↑ wiersze  
kolumny

char t[4][3] = {{1, 3, 5}, {2, 2, 4}, {10, 5, 19}, {3, 3, 93}}

## DYNAMICZNA ALOKACJA PAMIĘCI

```

typ *wsk;
wsk = (typ *) malloc(n * sizeof(typ));
// przydzielanie pamięci
free(wsk);
// zwalnianie pamięci
    
```

while(fscanf(...) > 0) == while(cin > x)

## WSKAŹNIKI CD.

- int \*const w ← trzeba ZAWSZE inicjalizować; innej BŁĄD
- funkcja(int tab[][4]); // deklaracja z wielowymiarową LUB (int (\*tab)[4]);
- \*w = NULL // nie wskazuje na nic

int main(int argc, char \*argv[])

↑  
liczba argumentów podanych z linii poleceń

↑  
tab parametrów podanych z linii poleceń

## WYJŚCIE

int printf(...); ← wypisuje i zwraca liczbę wypisanych znaków

|       |                                    |
|-------|------------------------------------|
| %d, i | INT                                |
| c     | CHAR                               |
| s     | CHAR* (STRING)                     |
| p     | VOID* (ADRES)                      |
| f, F  | FLOAT                              |
| e, E  | -11- NOTACJA WYKŁ.                 |
| u     | UNSIGNED INT                       |
| o     | ÓSEMKA                             |
| x, X  | SZESNASTKA                         |
| g, G  | WBIERZ CO LEPIEJ: FLOAT VS NOTACJA |

|         |                                                            |
|---------|------------------------------------------------------------|
| L       | LONG                                                       |
| 0       | wypełnij zerami (z lewej)                                  |
| liczba  | SZEROKOŚĆ POLA                                             |
| .liczba | LICZBA MŚC PO PRZECINKU [dla s LICZBA ZNAKÓW DO WYPISANIA] |
| -       | JUSTIFY DO LEWEJ (PRACU TO DEFAULT)                        |
| +       | WYMUSZA WYPISANIE ZNAKU (+ LUB -) LUBY                     |
| [space] | PRZED DODATNIMI WSTAWIA SPACJE (reby + i - były m równ)    |
| #       | dodaje 0 przed ósemkowym lub 0x/OX przed 16                |
| *       | SZEROKOŚĆ OKREŚLANA PRZEC ARGUMENT                         |
| L       | LONG DOUBLE                                                |

↑  
printf("%\*f", 10, a)  
printf("%10f", a)

## WEJŚCIE

int scanf(...);

PARAMETRY TO ADRESY!  
np: int a; scanf(..., &a)

PRZYKŁAD

\* S

PRZYKŁAD

stw

\* UN

~ typ



